

Authentication Methods Compared

Three ways to prove who you are when talking to an API — from simplest to most powerful.



SIMPLEST

API Key

A single secret string sent with every request. Like a password for your app.

HOW IT WORKS

1

PROVIDER

Gives you a secret key when you sign up

2

YOUR APP

Sends the key in a header with every request

```
Authorization: Bearer sk-abc123...
```

3

SERVER

Checks the key → grants or denies access



Easy — one string



If leaked, full access



Server-to-server, dev tools



MOST COMMON

JWT Token

Log in once, get a short-lived token. Like a wristband at a concert.

HOW IT WORKS

1

USER

Sends username + password to the login endpoint

2

SERVER

Verifies credentials, returns a signed token

```
# Token contains:  
user_id: 42 role: "admin" expires: 30 min
```

3

YOUR APP

Sends the token in every subsequent request

4

SERVER

Verifies the token's signature — no database lookup needed



Auto-expires (limits damage)



Carries user data inside



Web/mobile apps, your APIs

MOST POWERFUL

OAuth 2.0

Let users grant limited access via a trusted provider. Like a hotel giving a valet just the car key.

HOW IT WORKS

- 1 USER**
Clicks "Sign in with Google" on your app
- 2 PROVIDER (GOOGLE)**
Asks the user: "Allow this app to see your email?"
- 3 PROVIDER**
User approves → provider sends your app a limited-access token

```
scope: "email profile" # NOT "full account access"
```
- 4 YOUR APP**
Uses that token to access only what was approved

 User never shares password with you  Scoped permissions  Third-party login, integrations

Security Rules — Start Learning These Now

Never hardcode secrets

Use environment variables or .env files — never paste keys into source code.

Never commit keys to Git

Add .env to .gitignore immediately. One accidental push can expose everything.

Use HTTPS, always

HTTP sends tokens in plain text. HTTPS encrypts the connection so no one can intercept them.

Tokens expire for a reason

Short-lived tokens limit damage. If one is stolen, it stops working soon.

Least privilege

Request only the permissions you need. Don't ask for "full access" when "read email" is enough.

You'll implement JWT auth from scratch in [Module 5: FastAPI Development](#)